OpenBench OS - Full Product Specification (v2)

Bench Runtime Desktop Client + Web Control Console + Cloud Services

OpenAI

April 23, 2026

Document control

Document status: Draft for product, design, and engineering planning
Version: 2.0
Supersedes: Earlier demo-oriented PRD and PDF versions
Primary audience: Founder, product lead, engineering lead, frontend/backend engineers, design, safety reviewer, deployment/IT lead
Normative scope: This document defines the shipping product shape for V1 and the architectural decisions needed to build it. It is not a pitch deck and it is not a hackathon-only artifact.

Executive decision

OpenBench OS is **not** a single-surface demo app. It is a **full product platform** with three primary surfaces and one shared backend:

1. **Bench Runtime Client** - an installable desktop application for operators running procedures at the bench. Windows is the first-class packaged target in V1.
2. **Web Control Console** - a browser-based application for protocol upload, review, publishing, run oversight, admin, and reporting.
3. **Tablet / Mobile Companion** - a responsive installable web experience used mainly for photo capture, quick review, alerts, and handover consumption.
4. **Cloud Backend + AI Services** - document ingestion, protocol compilation, retrieval, run orchestration, photo verification, reporting, audit logging, and integrations.

One-sentence product definition

OpenBench OS is a protocol execution runtime for lab work that compiles approved SOPs, SDSs, equipment manuals, and lab rules into versioned execution graphs, then runs those graphs with explicit state, checkpoints, photo verification, deviation capture, citations, and handover reporting.

Product thesis

The product exists to solve one operational problem:

Approved lab procedures are hard to execute consistently in the moment of work.

The documents exist, but the runtime does not. Operators must translate dense SOPs, SDSs, and manuals into action while handling equipment, maintaining sterility/cleanliness, wearing PPE, and documenting deviations under time pressure. OpenBench OS provides the missing runtime layer between approved documentation and real execution.

What this specification changes versus the earlier PRD

The earlier PRD established the core thesis - protocol compiler, execution runtime, visual verifier, and handover historian - but it stayed too close to a demo format. This specification makes the product buildable by locking:

- product surfaces
- platform strategy
- technology stack
- role model
- service boundaries
- state model
- data model
- API surface
- offline/sync approach
- deployment topology
- security and privacy requirements
- test and release criteria

Product scope and success criteria

Product goals

V1 must:

- turn uploaded, approved documents into a structured and reviewable protocol graph
- publish immutable protocol versions
- allow an operator to execute a published version through a dedicated run engine
- persist run state outside the model
- support step-level questions with citations
- support note capture, deviations, timers, and confirmations
- support photo-based verification against step-specific visual checklists
- block or flag progress when required checkpoints are unresolved
- generate structured handover reports from run events
- provide manager/reviewer/admin workflows in a separate control console
- support Windows-first bench deployment with the same core UI model available on the web
- preserve auditability and source traceability for every major AI output

Non-goals for V1

The following are intentionally out of scope for the first shipping version:

- direct instrument control
- autonomous experiment design
- open-ended protocol generation from scientific intent
- real-time video monitoring
- identity recognition of people in photos
- biosafety or clinical decision support claims
- formal 21 CFR Part 11 compliance claims
- full ELN/LIMS parity
- generalized inventory management
- robotics orchestration

Success criteria

The product is successful when a lab can:

- upload a real procedure pack and publish a reviewed protocol version
- execute the procedure from a bench client without relying on PDF reading during the run
- catch at least one meaningful setup mismatch or documentation gap before work proceeds
- produce a structured run record and handover without reconstructing events after the fact
- review deviations and unresolved checkpoints centrally
- onboard new operators faster without sacrificing traceability

Product surfaces

Surface A - Bench Runtime Client

Purpose

This is the operator-facing application used during live execution.

Packaging decision

- **Form factor:** installable desktop application
- **Priority platform:** Windows-first
- **Technology:** shared React/Next.js UI packaged with Tauri 2
- **Secondary availability:** same UI capabilities exposed as a responsive web/PWA experience where desktop installation is not possible

Why this exists

A bench workflow benefits from an installable app because it offers:

- predictable access from shared lab workstations
- kiosk-like runtime behavior if needed
- easier local caching for offline or poor-network environments
- reduced dependency on the operator remembering a web URL or keeping browser state clean
- a dedicated product identity separate from the admin console

Core functions

- browse assigned/published protocols
- start and resume runs
- execute steps with progress state
- capture notes, confirmations, timers, and deviations
- ask step-specific questions
- capture/upload photos for checkpoints
- receive blocks, warnings, and escalation prompts
- generate or preview handover at end of run

Surface B - Web Control Console

Purpose

This is the reviewer/admin/manager surface.

Users

- protocol reviewers
- lab managers
- EHS/safety leads
- organization admins
- support staff for investigations and audits

Core functions

- upload and classify documents
- review extracted protocol graphs
- edit/normalize extracted steps and checkpoints
- publish and archive protocol versions
- manage users, permissions, and teams
- review runs, deviations, and blocked checkpoints
- review photo assessments and unresolved items
- manage retention, settings, and integrations
- export reports and audit records

Surface C - Tablet / Mobile Companion

Purpose

This is a lightweight responsive companion, not the primary authoring surface.

Core functions

- capture photos
- review current step
- confirm or acknowledge non-critical prompts
- receive escalation alerts
- review handover reports
- support roaming workflows where the operator cannot remain at a workstation

Surface D - Backend and AI Services

Purpose

This is the orchestration and persistence layer.

Core functions

- document ingestion
- OCR fallback
- chunking and indexing
- protocol graph generation
- hazard mapping
- Q&A retrieval and response
- photo verification
- report generation
- audit logging
- notifications
- integrations
- tenant isolation and permissions enforcement

Platform strategy

Product shape in plain language

OpenBench OS is **both**:

- a Windows-first installable bench application, and
- a web product with admin/reviewer surfaces

It is **not** a choice between “Windows app” or “website”. The shipping product is a multi-surface platform with shared UI and shared backend.

Why this is the right shape

A web-only product is weak for bench execution because it under-commits to local runtime, offline cache, and workstation deployment. A Windows-only native product is weak because protocol management, admin, and remote review are naturally browser-friendly and easier to deploy centrally.

The hybrid shape gives the product:

- dedicated runtime UX for operators
- centralized admin/review workflows
- shared frontend code
- deployability in mixed lab environments
- a clean path to macOS/Linux later if demanded

Offline and network strategy

The Bench Runtime Client must tolerate poor or intermittent connectivity. V1 therefore includes:

- encrypted local cache of published protocol versions
- local copy of active runs
- local event queue
- retry/sync service
- read-only access to already-synced source snippets needed for active runs
- upload retry for photos and audio if the network temporarily fails

The Control Console remains online-first.

Users, personas, and roles

Primary personas

Bench Operator

The operator executes procedures and needs short, context-aware instructions, fast data capture, visible checkpointing, and minimal screen clutter.

Protocol Reviewer

The reviewer approves compiled protocol graphs before they become runnable versions.

Lab Manager

The manager monitors team usage, deviations, blocked runs, and handover quality.

EHS / Safety Lead

The safety lead cares about hazard extraction quality, escalation logic, and the completeness of safety-critical checkpoints.

Organization Admin

The admin manages users, roles, retention, integrations, SSO later, and environment settings.

Role model

Operator

Can:

- view published protocols assigned to the operator's org/team
- start runs
- pause/resume runs
- add notes and deviations
- upload photos
- answer confirmations
- request override with reason
- generate handover

Cannot:

- publish protocol versions
- change published source requirements
- globally clear critical blocks without role-based permission

Reviewer

Can:

- review extracted protocol graphs
- edit draft versions
- publish protocol versions
- archive versions
- review flagged runs
- review override requests

Manager

Can:

- view all team runs
- approve certain override categories
- assign protocols
- export reports

- view analytics and unresolved issues

Safety Lead

Can:

- manage safety policy overlays
- review hazard mappings
- configure escalation taxonomy
- review safety-related deviations and photo evidence

Organization Admin

Can:

- manage memberships and roles
- configure retention and data region
- configure integrations
- manage allowed model providers
- view org-wide audit logs

Core user journeys

Journey 1 - Create and publish a runnable protocol

1. Reviewer uploads SOP, SDS, equipment manual, and optional policy documents.
2. System classifies, parses, chunks, and indexes source documents.
3. Protocol Compiler generates a draft protocol graph, hazard map, and step-level visual checklist candidates.
4. Reviewer inspects extraction confidence and edits any draft fields.
5. Reviewer publishes a version.
6. Published version becomes immutable and runnable.

Expected outputs

- protocol version ID
- normalized steps
- prerequisites and blockers
- required controls
- visual checkpoints
- citation map
- review trail

Journey 2 - Start and execute a run

1. Operator opens Bench Runtime Client.
2. Operator selects a published protocol version.

3. System runs a preflight check: document completeness, local sync status, required acknowledgements, pending safety notices.
4. Operator starts run.
5. Run engine advances step by step with explicit current state.
6. Operator logs notes, starts timers, captures values, and completes steps.
7. System blocks or warns when required conditions are unresolved.

Expected outputs

- run record
- event history
- timer events
- confirmations
- notes
- deviations

Journey 3 - Verify a visual checkpoint

1. Operator reaches a step with visual checks.
2. System displays only the relevant visual checklist for that step.
3. Operator uploads or captures a photo.
4. Vision Checkpoint Engine returns:
   - confirmed items
   - missing/not visible items
   - cannot verify items
5. Run engine updates blocker state.
6. If unresolved critical checks remain, the step cannot be completed without explicit override policy.

Journey 4 - Handle deviation or escalation

1. Operator logs a deviation or the system flags a stop condition.
2. Run transitions to warning, blocked, or escalated state depending on severity.
3. Operator may:
   - correct and continue
   - request manager/reviewer decision
   - stop run
4. All actions are appended as run events.

Journey 5 - Generate handover and review later

1. Operator ends run or pauses for shift handoff.
2. Report Generator builds handover from run events, notes, timers, measurements, and photo results.
3. Manager or next-shift operator reviews handover.
4. Report remains linked to exact protocol version and run record.

Functional modules

Module 1 - Document Ingestion

Summary

Ingest files, classify type, extract structured text, preserve page maps, and prepare them for compilation and retrieval.

Supported inputs in V1

- PDF
- DOCX
- image attachments for specific evidence capture
- optional CSV/JSON imports for future integration stubs

Requirements

- detect file type
- compute checksums
- preserve original file
- store page-level or chunk-level source references
- run OCR only when needed
- flag unreadable or low-quality scans
- support document version labels if present

Output artifacts

- stored source file
- parse metadata
- document chunks
- citation map
- extraction warnings

Module 2 - Protocol Compiler

Summary

Transform approved documentation into a structured protocol graph.

Compiler responsibilities

- identify protocol title, version, and scope
- extract ordered steps and branch conditions
- identify required materials and equipment
- identify timers, measurements, and expected observations
- identify visual-checkable items
- identify step prerequisites
- identify stop conditions

- emit structured JSON with citations and confidence scores

Reviewer controls

- edit titles and instructions
- merge/split steps
- mark required/optional checks
- remove unsupported extraction
- add reviewer notes
- publish immutable version

Module 3 - Hazard Mapping Service

Summary

Map SDS and policy material to step-level safety requirements.

Responsibilities

- parse SDS sections
- extract PPE requirements
- extract handling/storage/disposal constraints
- identify first-aid/spill/fire notes
- identify incompatibilities and escalation triggers
- attach source citations to hazard rules
- flag steps with missing SDS coverage

Module 4 - Run Engine

Summary

The Run Engine is the deterministic core of the product.

Responsibilities

- create runs from published protocol versions
- persist current step and progress
- enforce prerequisites and blockers
- manage timers
- attach notes and measurements to steps
- append all actions as events
- support pause/resume/reopen
- support override requests and outcomes

Design rule

The model is never the system of record for run state.
Run state lives in the database and local encrypted cache. The model reads state and returns typed outputs.

Module 5 - Execution Coach / Step Q&A

Summary

Contextual Q&A scoped to the current step or approved source set.

Rules

- answer only from approved source material or approved policy overlays
- always provide citations
- do not invent new procedures
- distinguish between “confirmed by source”, “inferred”, and “not verifiable”
- keep operator-facing answers terse by default
- surface escalation when the question seeks an unapproved substitution or modification

Module 6 - Vision Checkpoint Engine

Summary

Photo-based verification of step-specific visible conditions.

Rules

- check only items in the active visual checklist
- do not identify people
- do not infer hidden conditions
- use status taxonomy:
  - confirmed
  - not visible / missing
  - unclear
  - cannot verify
- request another image when the photo is insufficient
- persist evidence and rationale

Module 7 - Handover and Audit

Summary

Generate structured reports from events rather than from free-form chat memory.

Outputs

- run summary
- step completion timeline
- deviations
- unresolved items
- timer history
- photo evidence index
- citations for source-dependent claims

- review recommendations

Module 8 - Organization, Admin, and Policy

Summary

Tenant and environment management.

Capabilities

- user and team management
- role assignment
- protocol assignment
- data retention policies
- allowed model provider settings
- notification settings
- integration credentials
- audit access

Module 9 - Notifications

Channels

- in-app alerts
- email
- webhook
- optional Slack/Teams integration in later phase

Notification events

- run blocked
- run resumed
- override requested
- override approved/denied
- report ready
- protocol published
- protocol pending review

Module 10 - Integrations

V1 integration contract

V1 must provide:

- outbound webhooks
- CSV/JSON export
- REST API for read/write access to core entities
- object storage access via signed URLs internally

V2+ candidates

- ELN/LIMS export
- SSO / SCIM
- ticketing systems
- barcode and QR workflows
- instrument metadata ingestion

Information architecture and screen inventory

Bench Runtime Client navigation

Top-level navigation

- Home
- Protocols
- Active Runs
- Reports
- Settings
- Help / Safety Notes

Bench Runtime screens

1. Home

Shows assigned protocols, recently used runs, pending handovers, and blocked items requiring attention.

2. Protocol Library

Shows published protocol versions available to the operator.

3. Preflight Screen

Shows source completeness, local sync status, required acknowledgements, and start button.

4. Live Run Screen

Shows current step, progress, blockers, timers, current notes, and action buttons.

5. Photo Check Screen / Modal

Shows camera/upload flow, active checklist, results, and retake action.

6. Deviation Screen

Shows deviation form, severity, attachments, and submit action.

7. Handover Preview

Shows generated report before finalizing or sharing.

8. Offline / Sync Status Screen

Shows pending uploads, sync health, and retry options.

Web Control Console navigation

Top-level navigation

- Dashboard
- Protocols
- Runs
- Deviations
- Reports
- Team
- Settings
- Audit

Control Console screens

1. Dashboard

Run counts, blocked runs, pending reviews, recent deviations, and overdue handovers.

2. Upload Workspace

Document upload, classification, parse progress, and warnings.

3. Protocol Draft Review

Editable graph with source panes, confidence flags, and publish actions.

4. Published Version View

Version details, change history, assignment, archive action.

5. Run Detail View

Timeline, attachments, photo assessments, deviations, and handover report.

6. Deviation Queue

Filterable list of open deviations and unresolved checkpoints.

7. Team Admin

Users, roles, assignments, invitations, and org structure.

8. Settings

Retention, model provider settings, integration config, audit export.

Route map

Bench client routes

- /app/home
- /app/protocols
- /app/protocols/{protocolVersionId}
- /app/runs
- /app/runs/{runId}
- /app/runs/{runId}/checkpoint/{stepId}
- /app/runs/{runId}/deviation
- /app/runs/{runId}/handover
- /app/sync
- /app/settings

Web console routes

- /console/dashboard
- /console/protocols
- /console/protocols/new
- /console/protocols/{draftId}/review
- /console/protocols/{versionId}
- /console/runs
- /console/runs/{runId}
- /console/deviations
- /console/reports
- /console/team
- /console/settings
- /console/audit

Detailed functional requirements

Protocol lifecycle

States

- draft
- in_review
- published
- archived

Rules

- a draft can be edited
- a published version is immutable
- a run must always bind to one published version
- archived versions remain readable for audit and historical runs
- new uploads create a new version candidate rather than mutating history

Run lifecycle

States

- created
- preflight
- active
- paused
- blocked
- awaiting_override
- completed
- cancelled
- closed

Rules

- a run enters preflight before active
- a run enters blocked when a critical unresolved condition exists
- a run enters awaiting_override when a user requests permission to proceed past a critical blocker
- a run enters completed when all required terminal conditions are satisfied and the user finalizes handover
- closed is an admin/reviewer post-processing state if needed for investigations or archival workflows

Step lifecycle

States

- not_started
- in_progress
- waiting_on_timer
- waiting_on_checkpoint
- blocked
- completed
- skipped

Completion rules

A step can be marked complete only when:

- prerequisites are satisfied
- required confirmations are recorded
- required timers are satisfied or explicitly overridden
- required visual checks are either confirmed or overridden with justification
- no unresolved critical stop condition is active

Skip rules

- only steps marked skippable in the protocol may be skipped by operators
- all skips require reason capture
- non-skippable step skip requests become override events

Deviation handling

Every deviation must capture:

- run ID
- step ID if relevant
- title
- description
- severity
- timestamp
- actor
- attachments if any
- resolution state
- whether supervisor review is required

Escalation handling

Escalations are triggered by:

- source conflicts
- missing required source coverage
- unresolved critical visual mismatch
- user request for unapproved substitution
- hazard condition met
- user-reported exposure/spill/fire/injury/malfunction
- model unable to source answer for a critical instruction

Escalation outputs include:

- escalation type
- severity

- user-facing message
- required next action
- notify roles
- report inclusion flag

Role permissions matrix

Operator

- start run: yes
- edit draft protocol: no
- publish protocol: no
- upload photo: yes
- add note: yes
- add deviation: yes
- request override: yes
- approve own critical override: no by default
- export org-wide audit: no

Reviewer

- review draft protocol: yes
- publish protocol: yes
- archive protocol: yes
- review flagged runs: yes
- approve configured override categories: yes

Manager

- assign protocols: yes
- view team runs: yes
- approve configured override categories: yes
- export reports: yes

Safety Lead

- manage safety overlays: yes
- review escalations: yes
- configure escalation categories: yes

Org Admin

- manage users and settings: yes
- configure retention and integrations: yes
- access org audit exports: yes

AI system specification

AI responsibilities by pipeline

Pipeline A - Document understanding

Inputs: - uploaded documents - document metadata - parse quality signals

Outputs: - document type classification - normalized sections - chunk metadata - extraction confidence

Pipeline B - Protocol compilation

Inputs: - SOP - SDS - equipment manual - optional policy overlays

Outputs: - protocol graph JSON - hazard map JSON - step visual checklist candidates - confidence scores - source references

Pipeline C - Step Q&A

Inputs: - current step - current run state - approved source snippets - user question

Outputs: - answer text - citations - confidence - escalation flag - suggested next UI action

Pipeline D - Vision checkpoint

Inputs: - current step - visual checklist - supporting source snippets - image(s)

Outputs: - checklist item results - rationale - evidence notes - overall status - recommended next action

Pipeline E - Report generation

Inputs: - run events - deviations - photo assessments - timers - notes - protocol metadata

Outputs: - handover report - manager summary - structured JSON report object - PDF/HTML/Markdown render

Structured output requirement

All AI pipelines must return schema-validated JSON. Free-form text is allowed only in explicitly defined fields such as answer_text or summary_paragraph.

Retrieval policy

- retrieval scope defaults to the active protocol version and attached source pack
- Q&A may include approved org-level policy overlays if configured
- no retrieval from unrelated protocols unless the caller explicitly enters comparison mode and has permission
- citations must include source document ID and page/section or chunk ID

Safety reviewer pass

Every operator-facing AI response that could influence action is passed through a lightweight safety reviewer that checks:

- unsupported claims
- missing citations
- overconfident phrasing
- impermissible protocol invention
- escalation requirements
- forbidden identity/person analysis in photo responses

Confidence handling

Confidence is never used to silently suppress warnings. Instead:

- low confidence extraction is highlighted for reviewer attention
- low confidence photo results ask for another photo or manual confirmation
- low confidence Q&A can answer with “not enough source support” and escalate to source view or reviewer

Example schema - photo assessment


```
{
  "step_id": "step_004",
  "overall_status": "attention_required",
  "items": [
    {
      "check_id": "vc_1",
      "status": "confirmed",
      "evidence": "Two labeled tubes are visible in the front rack.",
      "confidence": "medium"
    },
    {
      "check_id": "vc_2",
      "status": "not_visible",
      "evidence": "Secondary containment tray is not visible in the frame.",
      "confidence": "medium"
    },
    {
      "check_id": "vc_3",
      "status": "cannot_verify",
      "evidence": "No readable airflow indicator is visible.",
      "confidence": "high"
    }
  ],
```

"recommended_action": "Do not complete the checkpoint until missing or non-verifiable required items are resolved."
}

State model and event model

Why events matter

OpenBench OS is built around append-only event capture so that:

- the report is derived from actual recorded actions
- investigators can reconstruct what happened
- offline synchronization has a durable source of truth
- protocol analytics can be computed later
- the model is not responsible for remembering run history

Event categories

- run_created
- run_started
- step_started
- step_completed
- timer_started
- timer_elapsed
- timer_overridden
- note_added
- measurement_recorded
- deviation_added
- checkpoint_requested
- photo_uploaded
- photo_assessed
- block_triggered
- override_requested
- override_resolved
- run_paused
- run_resumed
- run_completed
- handover_generated
- handover_finalized

Event requirements

Each event must include:

- event ID
- run ID
- event type
- actor ID
- device ID where relevant
- step ID if relevant
- timestamp
- payload JSON
- local sequence number for syncable client-originated events
- server receipt timestamp

Idempotency

Client-originated mutation requests must carry an idempotency key so retries do not duplicate events.

Data model

Design principles

- published protocol versions are immutable
- run events are append-only
- large files live in object storage
- structured records live in Postgres
- retrieval chunks are tied to exact document versions
- tenant isolation is enforced at the database access layer
- local desktop cache mirrors only what is required for assigned protocols and active/recent runs

Core entities

organizations

- id
- name
- slug
- data_region
- retention_policy_days
- created_at
- updated_at

users

- id
- email
- display_name
- status
- created_at
- updated_at

memberships

- id
- org_id
- user_id
- role
- team_id
- created_at

documents

- id
- org_id
- document_type
- title
- declared_version
- checksum
- storage_path
- mime_type
- parse_status
- ocr_status
- created_by
- created_at

document_chunks

- id
- document_id
- page_no
- section_label
- chunk_text
- embedding
- citation_json

protocols

- id
- org_id
- name
- status
- created_by
- created_at

protocol_versions

- id
- protocol_id
- version_label
- status
- source_docset_hash
- published_by
- published_at
- supersedes_version_id

protocol_steps

- id
- protocol_version_id
- step_key
- order_index
- title
- instruction
- is_skippable
- prerequisites_json
- required_ppe_json
- controls_json
- visual_checks_json
- stop_conditions_json
- expected_observations_json
- data_capture_schema_json
- source_refs_json
- confidence_score

hazard_rules

- id
- protocol_version_id
- step_id

- category
- requirement_text
- severity
- source_refs_json

runs

- id
- org_id
- protocol_version_id
- operator_id
- status
- current_step_id
- started_at
- ended_at
- device_id
- created_at

run_events

- id
- run_id
- event_type
- step_id
- actor_id
- device_id
- local_seq
- idempotency_key
- payload_json
- client_timestamp
- server_timestamp

step_state

- id
- run_id
- step_id
- status
- started_at
- completed_at
- blocked_reason_json

timers

- id
- run_id
- step_id
- label
- duration_seconds
- started_at
- ended_at
- status

deviations

- id
- run_id
- step_id
- severity
- title
- description
- resolution_state
- requires_review
- created_by
- created_at

attachments

- id
- run_id
- step_id
- kind
- storage_path
- mime_type
- checksum
- created_by
- created_at

photo_assessments

- id
- run_id
- step_id
- attachment_id
- overall_status
- items_json

- model_metadata_json
- created_at

handover_reports

- id
- run_id
- status
- report_json
- markdown_body
- html_body
- pdf_storage_path
- generated_at

audit_logs

- id
- org_id
- actor_id
- action
- target_type
- target_id
- metadata_json
- created_at

Local desktop cache

The Bench Runtime Client stores a limited local mirror using encrypted SQLite:

- published protocol versions assigned to the user/team
- active and recent runs
- queued offline events
- attachment upload manifest
- cached source snippets needed for active runs
- no full org-wide data replication

API specification

API principles

- REST for core CRUD and commands
- signed URLs for large object transfer
- server-generated IDs for durable entities
- idempotent mutation endpoints

- JSON schema contracts
- role-aware access control on every endpoint
- webhook delivery for external subscribers

Authentication and session endpoints

Method	Path	Purpose
POST	/api/auth/login	Sign in to web or desktop client
POST	/api/auth/refresh	Refresh session token
POST	/api/auth/logout	Revoke current session
GET	/api/auth/me	Current user and memberships

Document and protocol endpoints

Method	Path	Purpose
POST	/api/documents/upload	Create document upload and return signed upload target
POST	/api/documents/{id}/finalize	Finalize upload and trigger parsing
GET	/api/documents/{id}	Fetch document metadata
GET	/api/documents/{id}/chunks	Fetch document chunks/citations
POST	/api/protocol-drafts/compile	Compile a draft from a set of documents
GET	/api/protocol-drafts/{id}	Fetch compiled draft
PATCH	/api/protocol-drafts/{id}	Update draft fields during review
POST	/api/protocol-drafts/{id}/publish	Publish immutable protocol version
GET	/api/protocol-versions/{id}	Fetch published protocol version
POST	/api/protocol-versions/{id}/archive	Archive a version

Run endpoints

Method	Path	Purpose
POST	/api/runs	Create a run from a protocol version
GET	/api/runs/{id}	Fetch run summary and current state

Method	Path	Purpose
POST	/api/runs/{id}/start	Transition run to active
POST	/api/runs/{id}/pause	Pause run
POST	/api/runs/{id}/resume	Resume run
POST	/api/runs/{id}/steps/{stepId}/start	Start step
POST	/api/runs/{id}/steps/{stepId}/complete	Attempt completion with rule validation
POST	/api/runs/{id}/steps/{stepId}/skip	Skip step if permitted
POST	/api/runs/{id}/notes	Add note
POST	/api/runs/{id}/measurements	Add measurement
POST	/api/runs/{id}/deviations	Add deviation
POST	/api/runs/{id}/timers	Start timer
POST	/api/runs/{id}/override-requests	Request override

AI and checkpoint endpoints

Method	Path	Purpose
POST	/api/runs/{id}/ask	Step-scoped Q&A
POST	/api/runs/{id}/steps/{stepId}/photo-check	Upload photo and run checkpoint assessment
GET	/api/runs/{id}/steps/{stepId}/photo-assessments	List assessments for a step

Reporting endpoints

Method	Path	Purpose
POST	/api/runs/{id}/handover/generate	Generate report from event history
GET	/api/runs/{id}/handover	Fetch current report
POST	/api/runs/{id}/handover/finalize	Finalize handover
GET	/api/runs/{id}/handover/pdf	Download final PDF

Admin and audit endpoints

Method	Path	Purpose
GET	/api/admin/users	List users and roles

Method	Path	Purpose
POST	/api/admin/users/invite	Invite user
PATCH	/api/admin/memberships/{id}	Change role/team
GET	/api/admin/settings	Org settings
PATCH	/api/admin/settings	Update org settings
GET	/api/admin/audit	Query audit records
POST	/api/integrations/webhooks/test	Validate webhook target

Example command payload - run question

```
{
  "question": "What PPE applies here?",
  "context_mode": "current_step_only"
}
```

Example response payload - run question

```
{
  "answer_text": "This step requires eye protection, compatible gloves, and lab coat.",
  "citations": [
    {
      "document_id": "doc_01",
      "page_no": 3,
      "section_label": "Exposure controls / PPE"
    }
  ],
  "confidence": "high",
  "escalation_required": false,
  "suggested_action": "show_checklist"
}
```

Technology stack

Decision summary

Shared frontend

- **Framework:** Next.js App Router + React + TypeScript
- **Styling:** Tailwind CSS + shadcn/ui
- **State/query:** TanStack Query + Zustand
- **Validation:** Zod on the client, shared schema generation where useful

Desktop client

- **Shell:** Tauri 2
- **Local database/cache:** encrypted SQLite
- **Updater:** platform-specific signed updates
- **Native capabilities:** filesystem access boundaries, camera/mic bridge where needed, local notifications

Backend

- **API framework:** FastAPI
- **Schema layer:** Pydantic
- **ORM / migrations:** SQLAlchemy + Alembic
- **Background jobs:** Redis + Celery
- **Report rendering:** HTML/Markdown to PDF pipeline

Data and storage

- **Primary database:** PostgreSQL
- **Vector search:** pgvector
- **Object storage:** S3-compatible bucket storage
- **Tenant access control:** Row-level access policies in the database plus application checks

Observability and operations

- **Error tracking:** Sentry
- **Product analytics:** PostHog
- **Metrics/tracing:** OpenTelemetry + metrics backend
- **CI/CD:** GitHub Actions
- **Containers:** Docker for all backend services

Why this stack is selected

Next.js App Router

Selected because it supports a shared React web surface for the Control Console and responsive companion, while still allowing a coherent component model that can be reused inside the desktop packaging approach.

Tauri 2 for desktop

Selected because it lets the product package a shared web UI into a dedicated desktop application rather than requiring an entirely separate native UI stack.

FastAPI

Selected for typed Python APIs, straightforward background integration, JSON-schema-friendly contracts, and proximity to document and AI processing code.

PostgreSQL + pgvector

Selected because the product needs transactional state, auditability, JSON support, and vector retrieval in one system.

Redis + Celery

Selected because document parsing, compilation, photo assessment, and report generation are asynchronous jobs that should not block request-response UX.

SQLite local cache

Selected because the operator app needs resilient local caching and event journaling for intermittent network conditions.

Alternatives considered

Electron instead of Tauri

Rejected for V1 because the product benefits from a lighter desktop shell and a clearer separation between shared web UI and minimal native wrapper.

Node/Express backend instead of FastAPI

Rejected because the most complex backend workloads - document parsing, OCR fallbacks, retrieval orchestration, image assessment, and report generation - are already naturally Python-centric.

Browser-only web app instead of packaged desktop client

Rejected because bench workflows need local caching, installability, controlled workstation deployment, and a dedicated operator runtime.

Native Windows-only frontend

Rejected because it would slow development, fragment the UI codebase, and weaken the web admin/reviewer surface.

Architecture

Logical architecture

The runtime architecture is organized in six layers:

1. **Client layer**
   - Bench Runtime Client (desktop, Windows-first)
   - Web Control Console
   - Responsive companion / PWA surface
2. **Gateway layer**

    - FastAPI API Gateway
    - auth/session validation
    - signed URL generation
    - request validation and command routing
3. **Core application services**
    - Protocol Service
    - Run Orchestrator
    - Report Service
    - Notification / Integration adapters
4. **AI and asynchronous workers**
    - document parsing and OCR fallback
    - protocol compilation
    - step Q&A
    - vision checkpoint assessment
    - report assembly jobs
5. **Persistence layer**
    - PostgreSQL + pgvector
    - Redis job queue and cache
    - object storage for files and report artifacts
6. **Operations layer**
    - audit logging
    - metrics
    - tracing
    - backup and restore

Service boundaries

API Gateway

- auth
- request validation
- signed URL generation
- command routing
- response aggregation

Protocol Service

- document metadata
- parse state
- draft compilation lifecycle
- published version management

Run Orchestrator

- run state transitions
- step completion rules

- timer handling
- override request flow
- event append

AI Worker Pool

- parsing fallback jobs
- protocol compilation jobs
- step Q&A
- vision assessments
- report generation

Report Service

- report JSON assembly
- markdown/html render
- PDF generation
- report storage and export

Sync Service

- desktop client event ingestion
- conflict detection
- attachment upload reconciliation

Deployment topology - SaaS default

- frontend web assets on managed hosting/CDN
- API service in containers
- worker service in containers
- managed Postgres
- managed Redis
- managed object storage
- per-environment isolation: dev, staging, prod

Deployment topology - self-hosted / dedicated VPC

- same containerized services
- customer-managed Postgres/Redis/object storage equivalents
- private networking
- SSO and secret management via customer environment
- outbound AI provider access mediated by customer policy

Offline and synchronization design

Scope

Offline support is required for the Bench Runtime Client, not for every Control Console feature.

What works offline

- open cached assigned protocols
- resume active cached runs
- read step instructions and citations already synced locally
- capture notes, deviations, timer events, and confirmations
- capture photos for later upload
- queue override requests
- generate provisional local handover preview from local event history

What does not work fully offline

- publishing new protocol versions
- org admin changes
- fetching uncached protocol history
- sending notifications to remote users
- reviewer decisions that require server authority

Sync model

Local event journal

Every client-side mutation is first written locally with:

- local event ID
- local sequence number
- idempotency key
- pending/synced/error status

Server reconciliation

When connectivity exists:

- queued events are posted in order
- server acknowledges and maps them to canonical records
- attachments are uploaded via signed URLs
- any rejected events are marked with reason and surfaced to the operator

Conflict handling

Conflict resolution follows these rules:

- published protocol versions never conflict because they are immutable
- run events are append-only; two valid events can coexist
- conflicting step outcomes become a reviewable inconsistency instead of silent overwrite
- last-write-wins is allowed only for non-critical local UI preferences, never for run history

Security, privacy, and compliance posture

Security goals

- protect proprietary procedures and attachments
- prevent cross-tenant access
- preserve auditability
- reduce prompt injection risk from uploaded documents
- protect operator credentials and local caches
- avoid unsupported safety claims

Authentication and authorization

V1 includes:

- email/password or SSO-ready auth provider integration
- per-org memberships
- role-based access control
- database-enforced row isolation
- short-lived access tokens with refresh flow
- signed upload/download URLs for objects

Data protection

- TLS in transit
- encryption at rest for database and object storage
- encrypted local SQLite database for the desktop cache
- secrets stored in a dedicated secrets manager per environment
- configurable retention and deletion windows

Privacy promises

- customer documents are not used to train foundation models
- data export and deletion workflows exist
- attachments and reports inherit tenant access control
- optional redaction support for names/sample IDs in exports later

Audit requirements

Audit logs must capture at least:

- who uploaded or changed a protocol draft
- who published a version
- who started or ended a run
- who approved or denied an override
- who changed org settings
- who exported sensitive reports

Compliance posture

V1 is built to be compliance-aware, not compliance-certified. The product may borrow concepts such as immutable histories, role checks, time-stamped actions, and record-linking, but it must not market itself as formally compliant with regulated-record standards until validated accordingly.

Accessibility specification

Accessibility principles

- hands-busy workflows are a first-order use case
- important actions must be accessible without precise pointer control
- every spoken response must be available as visible text
- color cannot be the only status signal
- critical information must remain readable with gloves, distance, and time pressure in mind

V1 requirements

- keyboard navigable primary workflows
- high contrast theme
- large target mode
- readable text scaling
- one-step-at-a-time mode
- text-to-speech for current step
- repeat-last-instruction action
- “critical only” condensed view
- captions/transcripts for voice features
- semantic labels for assistive technology
- accessible summary of photo-check results, not image-only feedback

Voice feature posture

Voice is an enhancement layer, not the sole control path. Every voice action must have an equivalent touch/keyboard action.

Performance and reliability targets

Operator UX targets

- open cached protocol: under 2 seconds
- start run from already-synced protocol: under 3 seconds
- step transition: under 1 second for local state update
- Q&A response: p95 under 5 seconds

- photo assessment response: p95 under 15 seconds
- report generation: p95 under 30 seconds
- local note/deviation capture: under 500 ms

Backend SLOs

- API availability: 99.9% monthly target for hosted production
- job success rate: 99% with retry policy for transient failures
- durable event persistence before user-facing success on any mutation
- no silent data loss on client reconnect

Error handling requirements

- failed AI response cannot lose operator-entered data
- photo upload failure must preserve local file and retry state
- every hard failure must produce a human-readable recovery path
- unresolved sync errors remain visible until cleared

Observability and analytics

Engineering observability

Track:

- request latency
- job queue depth
- protocol compile failures
- photo assessment failures
- report generation failures
- sync retry counts
- client offline duration
- desktop app version distribution

Product analytics

Track:

- time from upload to publish
- time from run start to handover
- note/deviation rates per protocol
- blocked run frequency
- override request frequency
- photo retake frequency
- unresolved checkpoint rate
- feature adoption by role

Privacy guardrails for analytics

- do not send source document content to product analytics
- minimize personally identifying data
- use event/property design that supports tenant-safe analysis

Testing and validation strategy

Test layers

Unit tests

- protocol rule functions
- state transitions
- API schema validation
- local sync logic
- report assembly

Integration tests

- upload -> parse -> compile flow
- publish -> start run -> complete run flow
- photo assessment integration
- offline queue -> reconnect sync flow
- signed URL upload/download flow

End-to-end tests

- reviewer publishes protocol
- operator executes run
- manager reviews handover
- override request and resolution
- blocked checkpoint workflow

AI evaluation

- golden protocol extraction set
- golden photo assessment set
- Q&A citation coverage set
- safety refusal/escalation prompts
- hallucination regression set

Load tests

- concurrent compile jobs
- concurrent run updates
- burst photo uploads
- report generation spikes

Release gates

A release cannot ship unless:

- critical path end-to-end tests pass
- schema migrations are reversible
- no P0 or unmitigated P1 security issues remain
- golden AI evaluation stays above agreed thresholds
- desktop upgrade path is verified from previous stable version

Release roadmap

Phase 0 - Foundation

- auth, orgs, roles
- document ingestion
- protocol drafts
- publish immutable versions
- core run engine
- desktop shell
- basic report generation

Phase 1 - Full V1 shipping scope

- photo checkpoint engine
- offline queue and sync
- deviation and override workflows
- reviewer/manager console
- audit log
- PDF export
- analytics and monitoring
- accessibility baseline

Phase 1.1

- richer team dashboards
- improved OCR and label reading
- more robust handover templates
- notification routing
- better admin controls

Phase 2

- SSO / SCIM
- ELN/LIMS integrations
- policy overlays per org/team

- advanced audit exports
- improved self-hosted deployment package
- structured training and certification links

Phase 3

- validated workflows for regulated environments
- e-signature framework
- deeper equipment metadata
- change-control and review workflows
- advanced integration marketplace

Implementation plan by team

Frontend / desktop team

Owns:

- shared component library
- bench runtime screens
- web console screens
- Tauri desktop shell
- local cache and sync UX
- accessibility and responsive behaviors

Backend team

Owns:

- API contracts
- protocol service
- run orchestrator
- storage integration
- authz hooks
- report service
- notifications
- audit service

AI / document intelligence team

Owns:

- parsing/OCR workflow
- protocol compiler
- hazard mapping
- retrieval and citation assembly

- photo assessment
- safety reviewer
- evaluation harness

DevOps / platform team

Owns:

- environments
- CI/CD
- secrets
- observability
- backups
- deployment automation
- desktop release signing/update pipeline

Risks and mitigations

Risk 1 - The product feels like an LLM wrapper

Mitigation: make the protocol graph, run engine, blockers, event model, and immutable versioning first-class product primitives. Chat is a surface, not the product.

Risk 2 - Unsafe or overconfident AI output

Mitigation: schema-validated outputs, mandatory citations, safety-review pass, and refusal/escalation for unsupported instructions.

Risk 3 - Offline complexity delays delivery

Mitigation: ship bounded offline scope focused on assigned protocols and active runs, with append-only local event journaling rather than broad local mirroring.

Risk 4 - Reviewer burden is too high

Mitigation: confidence scoring, change highlighting, reusable review presets, and import normalization to reduce manual cleanup.

Risk 5 - Photo verification becomes unreliable

Mitigation: restrict checks to visible checklist items, avoid hidden-state inference, add retake flow, and design explicit “cannot verify” states.

Risk 6 - Protocol versioning becomes messy

Mitigation: immutable published versions, clear supersession rules, and runs pinned to exact versions.

Risk 7 - Multi-surface product creates design drift

Mitigation: one shared design system, one shared schema library, and common state semantics across desktop/web.

Out of scope register

The following remain outside V1 even if technically possible:

- direct control of lab instruments
- human identity recognition or compliance judgment from images
- protocol generation from a scientific goal prompt
- always-on video analysis
- replacing local lab emergency procedures
- medical or clinical decision support
- certification of safety or regulatory compliance
- general inventory/warehouse management

Open questions for kickoff

These questions should be resolved early in delivery planning:

1. Which customer segment is the first beachhead: academic labs, biotech R&D teams, or industrial QC labs?
2. Is Windows-first sufficient for the first 2-3 pilot customers, or is macOS packaging required on day one?
3. Which override categories can managers approve versus safety reviewers?
4. What retention defaults are acceptable for attachments and photo evidence?
5. Which integration path matters first: webhook-only, CSV export, or direct ELN push?
6. What minimum offline duration must the desktop client support without re-authentication?
7. Which model providers are allowed in hosted versus self-hosted deployments?

Final product summary

OpenBench OS V1 is a **full application platform**, not a demo. It consists of:

- a Windows-first installable Bench Runtime Client for operators
- a browser-based Web Control Console for reviewers, managers, and admins
- a responsive companion experience for tablet/mobile workflows
- a cloud backend with document intelligence, run orchestration, photo verification, reporting, and audit services

The defining architectural choices are:

- immutable published protocol versions
- explicit run state machine
- append-only event history
- local desktop cache plus sync queue
- AI outputs constrained by typed schemas and citations
- photo verification scoped to step-specific checklists
- reporting derived from events, not chat memory

This is the build-ready shape of the product.

Appendix A - Recommended repository layout

```
/openbench
  /apps
    /console-web       # Next.js control console
    /bench-desktop     # Tauri shell reusing shared frontend
  /packages
    /ui                # shared components
    /schemas           # shared Zod / JSON schemas
    /client-core       # sync client, storage abstractions
  /services
    /api               # FastAPI gateway and domain services
    /workers           # background jobs and AI pipelines
  /infra
    /docker
    /k8s
    /migrations
    /scripts
  /docs
    /product
    /architecture
    /runbooks
```

Appendix B - Reference implementation notes for stack decisions

These references are informational and support the platform choices in this specification. The PRD itself remains the normative product document.

1. Tauri 2 documentation - supports existing web stacks and packaging/distribution for desktop installers.
2. Next.js App Router documentation - shared React application structure with file-system routing and modern React patterns.

3. FastAPI deployment documentation - container-based deployment patterns and worker/process guidance.
4. Supabase/Postgres Row Level Security documentation - tenant-safe access patterns and database-level protection.
5. Supabase pgvector documentation - vector search on top of Postgres for retrieval use cases.
6. MDN MediaDevices/getUserMedia documentation - camera/microphone access and secure-context requirements for web/PWA modes.
7. web.dev PWA installation guidance - installability expectations for responsive companion mode.
8. WCAG 2.2 - accessibility baseline for the web surfaces.

Appendix C - Source basis

This document supersedes the earlier OpenBench OS materials and expands them into a build-ready full-app specification:

- Earlier OpenBench OS PRD (PDF)
- Earlier OpenBench OS PRD (Markdown)